Software Development & Testing Fundamentals

AI-Based Test Automation Course Notes

Session Plan & Topic Flow

Order	Topic	Focus Area	Est. Time
1	**Introduction**	Why Testing and Automation Matter	5 mins
2	**SDLC (Software Development Life Cycle)**	Phases of building software	10 mins
3	**Agile, Scrum & Agile Team Structure**	Sprints, ceremonies, and team roles (PO, SM, QA, Dev)	15 mins
4	**STLC (Software Testing Life Cycle)**	Structured parallel testing process	10 mins

Order	Topic	Focus Area	Est. Time
5	**QA Roles & Responsibilities**	QA vs. QC (Prevention vs. Detection)	5 mins
6	**Requirement Analysis & Acceptance Criteria**	Turning requirements into Gherkin (Given-When-Then)	10 mins
7	**Test Scenario vs. Test Case**	Structuring *what* to test vs. *how* to test	10 mins
8	**Test Case Design Techniques**	Equivalence Partitioning (EP) & Boundary Value Analysis (BVA)	10 mins
9	**Testing Levels & Types**	Functional, Integration, System (E2E), and UAT	15 mins
10	**Test Execution & Stability Suites**	Smoke, Sanity, Regression, and Exploratory Testing	15 mins
11	**Defect Lifecycle & Management**	Bug states, Severity vs. Priority, and Root Cause Analysis (RCA)	15 mins

Total Duration: 2 Hours (120 Minutes)

Learning Objectives

By the end of this session, you will be able to:

- Map testing activities directly to SDLC, STLC, and Agile Sprint phases.
- Understand how cross-functional Agile teams are formed and the role of QA in Sprints.
- Write clear, professional test cases and test scenarios from business requirements using EP and BVA.
- Classify testing types (Smoke, Sanity, Regression, Integration, System, UAT) and apply them to target features.
- Log, categorize, and track bugs through the Defect Lifecycle, and use Severity vs. Priority matrices to triage them.

1. Introduction

Before writing automation scripts in Playwright, you must master software testing fundamentals. Automated tests are only as valuable as the manual test strategies behind them.

Knowing **why** we test, **how** software is built (SDLC), **how** cross-functional teams collaborate (Agile), and **how** quality is tracked guarantees that your Playwright scripts target high-value, high-risk user paths rather than redundant checks.

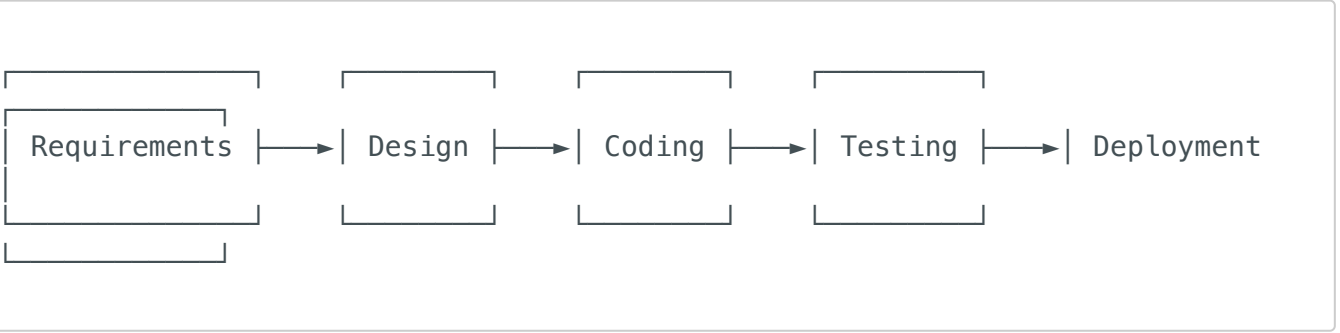
---

2. SDLC (Software Development Life Cycle)

Definition

The **Software Development Life Cycle (SDLC)** is a structured engineering process used by software teams to plan, design, build, test, deploy, and maintain high-quality software systems. *(Software banane, design karne, test aur deploy karne ka systematic sequence.)*

Core Phases



1. **Requirement Analysis:** Gathering business goals and deciding *what* to build.
2. **Design:** Creating system architecture, UI mockups, and database structures.
3. **Coding (Implementation):** Developers writing code (e.g., React frontend, Node backend APIs).

4. **Testing:** QA verifying the product against original specifications.
5. **Deployment:** Launching the application onto Staging or Production servers.
6. **Maintenance:** Monitoring app performance, fixing bugs, and handling upgrades.

💡 Sarathi Store Example

Building a new **Coupon Code feature** for the checkout page:

- **Requirements:** Product Owner decides the store must accept coupon codes like `SAVE10` to apply percentage discounts on checkout.
- **Design:** DB architects design a `coupons` table structure (code, type, value, expiry, min_purchase).
- **Coding:** Backend developers write `couponController.js` and frontend developers add the coupon field to `Checkout.tsx`.
- **Testing:** QAs test valid, invalid, and expired coupons using different cart values.
- **Deployment:** The code is merged and deployed to the staging environment.
- **Maintenance:** Monitoring user logs for checkout failures related to invalid discounts.

Common Pitfalls & Real-World Usage

> [!WARNING] **Delayed Testing (The "Waterfall" Trap):** If QA only gets involved in the Testing phase, bugs introduced during the Requirements or Design phases become 10x more expensive to fix. Early testing is crucial to prevent waste.

---

3. Agile, Scrum & Agile Team Structure

Definition

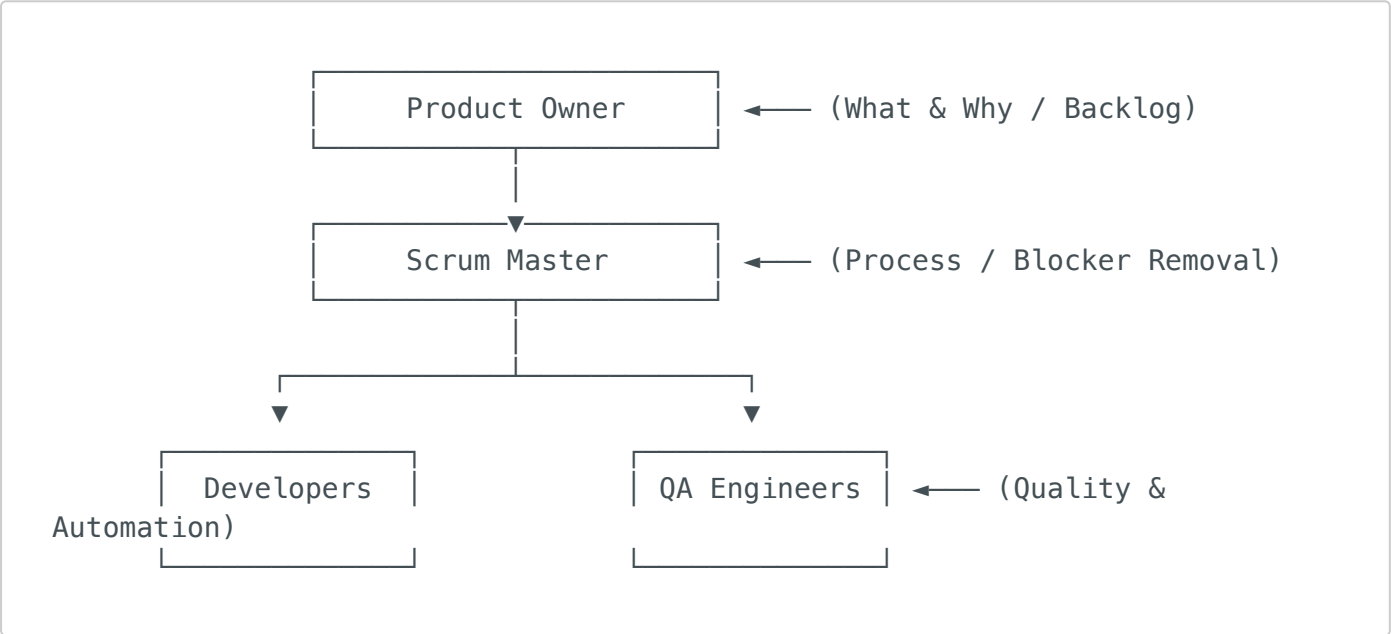
- **Agile** is a software development philosophy promoting iterative delivery, flexibility, and customer feedback.
- **Scrum** is a specific framework implementing Agile through fixed-length cycles called **Sprints** (usually 2 weeks).

How Agile Teams are Formed

Agile teams are formed as **cross-functional, self-organizing pods**. Rather than keeping developers, QAs, and designers in separate departments (silos), they are grouped together into a single unit dedicated to a specific product area or set of features (e.g., "Checkout & Payments pod").

- **Team Size:** Ideally 5 to 9 members ("Two-Pizza Team" rule to optimize communication).
- **Self-Organizing:** The team decides *how* to accomplish the work defined by the business, rather than having manager-driven top-down commands.

Key Roles in an Agile/Scrum Team



1. **Product Owner (PO):**
    - Represents the customer/business.
    - Defines the **"What" and "Why"** of the product.
    - Manages and prioritizes the Product Backlog, writes User Stories, and defines Acceptance Criteria.
2. **Scrum Master (SM):**
    - Acts as a process coach and facilitator.
    - Focuses on removing roadblocks (blockers) for the team and protecting them from external disruptions.
    - Ensures Scrum ceremonies are run effectively.
3. **Developers (Frontend, Backend, DevOps):**
    - Build the product features.
    - Responsible for writing clean code, designing database schemas, creating API endpoints, and writing Unit Tests.

4. **QA Engineers:**
    - **Quality Advocates:** Help the team think about edge cases and testability before coding starts.
    - Write and execute test scenarios, report defects, verify bug fixes, and design browser automation frameworks (e.g., Playwright).

How QA Operates in Agile Sprint Ceremonies

In a 2-week Sprint, testing is a continuous activity, not an afterthought:

- **Sprint Planning:**
    - The team selects user stories from the prioritized backlog.
    - *QA Action:* QA reviews user stories for testability, asks clarifying questions about edge cases, and estimates the testing effort (testing points).
- **Daily Standup (15-min daily sync):**
    - The team aligns on: *What did I do yesterday? What will I do today? Are there any blockers?*
    - *QA Action:* QA reports progress (e.g., "*I finished writing test cases for Checkout; waiting for build deployment to test*").
- **Sprint Review (Demo):**
    - The team shows working software built during the Sprint to stakeholders.
    - *QA Action:* QA assists in showcasing stable features or highlighting quality achievements.
- **Sprint Retrospective:**
    - Held at the end of the sprint to discuss: *What went well? What didn't? How can we improve?*
    - *QA Action:* QA raises process improvements (e.g., "*API specifications were modified mid-sprint without updating QAs, which broke our automation drafts. We should lock API contracts early.*")

---

4. STLC (Software Testing Life Cycle)

Definition

The **Software Testing Life Cycle (STLC)** is a systematic sequence of testing activities executed during the development lifecycle to verify that software meets quality standards. *(STLC aur SDLC parallel chalte hain. SDLC code likhne ke liye hai, STLC use verify karne ke liye.)*

STLC Phases, Entry/Exit Criteria, and QA Activities

```
Requirement Analysis ⟶ Test Planning ⟶ Case Design ⟶ Env Setup ⟶
Execution ⟶ Closure
```

Phase	QA Activities	Entry Criteria	Exit Criteria
1. Requirement Analysis	Review requirements and UI designs; identify testable features.	Business Requirements Document (BRD) / User Stories available.	Requirements signed off; testable queries resolved.
2. Test Planning	Define test scope, resource needs, timelines, and test environment details.	Requirement analysis completed.	Test Plan approved; tool selection finalized (e.g., Playwright).
3. Test Case Development	Write detailed Test Cases, Test Scenarios, and prepare test data.	Test Plan ready.	Test Cases written, peer-reviewed, and approved.
4. Environment Setup	Prepare testing database, server endpoints, and automation runners.	Test cases and test data defined.	Testing environment (Staging/QA) is up, running, and accessible.
5. Test Execution	Run tests (manual + automation), log bugs, and retest bug fixes.	Environment ready; Test cases available.	All test cases executed; no blocker/critical bugs remain open.

Phase	QA Activities	Entry Criteria	Exit Criteria
6. Test Closure	Analyze test results, metrics (pass/fail ratio), and publish summary reports.	Test execution completed.	Test Summary Report generated and signed off by stakeholders.

💡 Sarathi Store Example

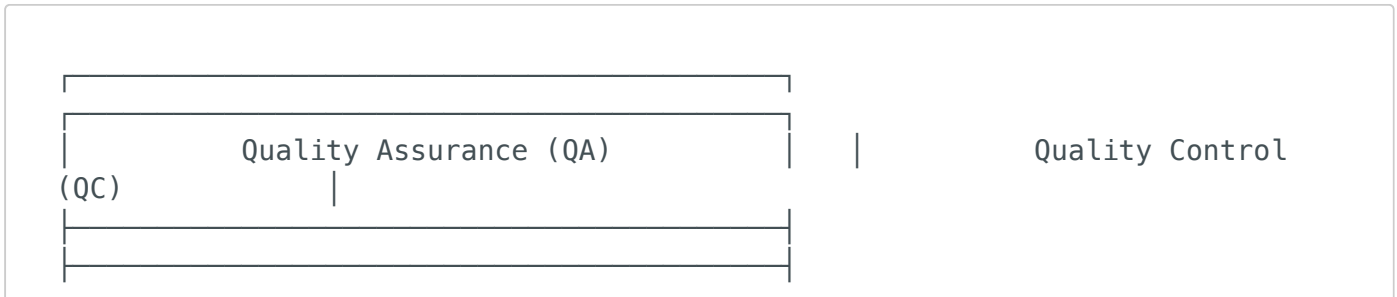
Executing STLC for the **Customer Login Page** (`frontend/src/pages/Login.tsx`):

- **Requirement Analysis:** Read the user story: Login requires valid credentials, redirects Regular Customers to `/`, and shows demo credentials in a table at the bottom of the page.
- **Test Planning:** Plan to use Playwright for cross-browser testing (Chrome, Safari, Firefox) on both desktop and mobile viewports.
- **Test Case Development:** Write test cases (e.g., login with valid customer, login with invalid email format, assert demo credentials exist).
- **Environment Setup:** Configure the backend server running locally on `http://localhost:5000` with Supabase seed data and launch Vite frontend.
- **Test Execution:** Execute manual checks first, then run automated Playwright test scripts. Report any redirection issues as bugs.
- **Test Closure:** Document test pass percentage (e.g., 90% passed, 1 minor bug deferred).

---

5. QA Roles & Responsibilities (QA vs. QC)

It is critical to distinguish between Quality Assurance (preventative) and Quality Control (detective).



```
| • Focus: Process-oriented                    |   | • Focus: Product-oriented
|
| • Goal: Prevent defects from entering        |   | • Goal: Detect defects in
the build      |
| • Action: Reviews requirements, code         |   | • Action: Executes test
cases, files     |
|   validation logic, designs workflows.       |   |   bugs, performs
manual/auto runs.       |
| • Proactive                                  |   | • Reactive
|
└──────────────────────────────────────────────┘
└──────────────────────────────────────────────┘
```

💡 Sarathi Store Examples

- **QA (Prevention):** A QA engineer reviews the React State management script (`CartContext.tsx`) and notices that `Number()` parsing is done for `price` and `stock` fields on line 63-68:

  ```
  quantity: Number(item.quantity),
  product: { ...item.product, price: Number(item.product.price) }
  ```

  QA verifies with developers that this prevents issues with string-type calculations which would cause NaN errors on the cart checkout page, preventing bugs before coding wraps up.
- **QC (Detection):** Running Playwright scripts to add two items costing `$10.50` each to the shopping cart and asserting that the UI displays a cart total of `$21.00`.

---

6. Requirement Analysis & Acceptance Criteria

Requirement Analysis

Requirements are often vague or incomplete. Requirement Analysis translates vague requests into testable conditions.

- **Vague Requirement:** "*The Sarathi Store product search must load quickly and display relevant results.*" (Untestable: What does "quickly" mean? What makes results "relevant"?)
- **Testable Requirement:** "*When a user types 'iPhone' in the Home page search input and presses Enter, the page must fetch results from* `/api/products?search=iPhone` *within 1.5 seconds and display matching cards containing the product image, title, price, and 'Add to Cart' button.*"

Acceptance Criteria (AC)

Acceptance Criteria define the boundary rules of a user story. They are commonly written in the Gherkin format to ensure clarity:

- **Given:** Preconditions (Context)
- **When:** User Action (Event)
- **Then:** Expected Outcome (Consequence)

💡 Sarathi Store Example

User Story: Applying discount coupons at checkout.

- **AC 1: Minimum Cart Value Restriction**
  - **Given** the customer is logged in and has items in their cart totaling `$80.00`
  - **And** the coupon code `SAVE10` requires a minimum purchase of `$100.00`
  - **When** the customer enters `SAVE10` and clicks the "Apply" button
  - **Then** the checkout page should display an error alert: "*Minimum purchase amount required: $100.00*"
  - **And** the discount amount should display `$0.00`.

---

7. Test Scenario vs. Test Case

Understanding the difference between high-level scenarios and detailed test cases keeps your test suite structured and maintainable.

```
Test Scenario (What to test)
  └─ Test Case 1 (How to test – Positive Path)
  └─ Test Case 2 (How to test – Negative Path)
  └─ Test Case 3 (How to test – Boundary Case)
```

- **Test Scenario:** High-level identification of what to test. Usually a single line summarizing a user workflow.
- **Test Case:** A detailed set of preconditions, step-by-step instructions, test data inputs, and expected outcomes verifying specific behaviors.

💡 Sarathi Store Example

- **Test Scenario:** Verify the customer checkout checkout page validation.
- **Test Cases mapped to this Scenario:**
  1. **TC-CHECKOUT-01 (Positive):** Complete checkout with valid shipping address and payment inputs.
  2. **TC-CHECKOUT-02 (Negative):** Attempt checkout with a blank ZIP/Postal code (verify warning message).
  3. **TC-CHECKOUT-03 (Edge Case):** Checkout with cart items exceeding current stock inventory (verify inventory rejection error).

---

8. Test Case Design Techniques

Test case design techniques allow you to select the most effective inputs from thousands of possibilities, maximizing test coverage while keeping execution fast.

1. Equivalence Partitioning (EP)

EP divides the input data of a program into partition classes from which test cases can be derived. Input data within a partition behaves identically.

- **Valid Partition:** Inputs that should be accepted.
- **Invalid Partition:** Inputs that should be rejected with an error.

2. Boundary Value Analysis (BVA)

BVA focuses on the boundaries of equivalence partitions. System errors most frequently occur at the boundary limits of input domains.

- Test values are selected at: **Boundary**, **Just Below Boundary (Boundary - 1)**, and **Just Above Boundary (Boundary + 1)**.

💡 Sarathi Store Example

Looking at the coupon validation logic inside `couponController.js` (lines 145-153):

```
const minAmount = parseFloat(coupon.min_purchase_amount); // e.g., $100.00
const totalAmount = parseFloat(cart_total);
if (totalAmount < minAmount) { ... reject coupon ... }
```

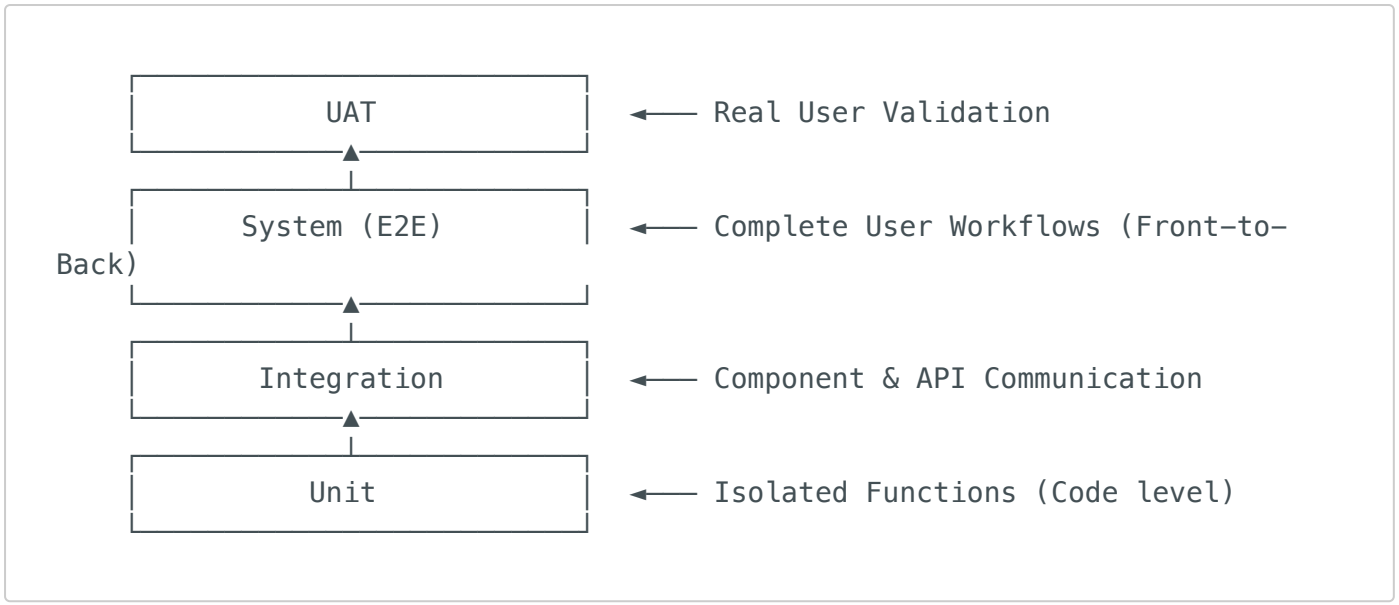
Let's design test cases using EP and BVA for a coupon requiring a minimum purchase amount of **`$100.00`**:

- **Equivalence Partitions (EP):**
  - **Valid Partition:** `[100.00 to Infinity)` (e.g., cart total of `$150.00` should successfully apply coupon).
  - **Invalid Partition:** `[0.00 to 99.99]` (e.g., cart total of `$50.00` should reject coupon with warning).
- **Boundary Value Analysis (BVA) inputs:**
  1. **`$99.99`** (Just below boundary - Invalid partition. Coupon should fail).
  2. **`$100.00`** (On the boundary - Valid partition. Coupon should apply).
  3. **`$100.01`** (Just above boundary - Valid partition. Coupon should apply).

---

9. Testing Levels & Types

Software testing is categorized into distinct levels of isolation and system integration.



1. Functional Testing

Black-box testing of UI screens, buttons, inputs, and pages without inspecting internal server source code.

- 💡 **Sarathi Store Example:** Testing the "Product Search" bar by searching for `"Computer"` and verifying that matching cards display correct descriptions.

2. Integration Testing

Verifies that separate modules, databases, and microservices interact and exchange data correctly.

- 💡 **Sarathi Store Example:** Testing the interaction between frontend component hooks calling backend paths.
  - **Flow:** `CartContext.tsx` invokes `api.post('/cart/items', { product_id, quantity })` ⟶ Backend routes to `/api/cart/items` in `routes/cart.js` ⟶ Inserts into Supabase `cart_items` table.
  - **Verification:** Assert that the backend database values update correctly upon calling the API endpoint.

3. System Testing (End-to-End / E2E)

Testing the complete, fully integrated software application from start to finish to ensure logical flows execute smoothly.

- 💡 **Sarathi Store Example:**
  - **E2E Flow:**
    1. **Customer** logs in, searches for a product, adds it to the cart, applies a discount coupon, and places an order.
    2. **Database** updates the order status to `pending`.
    3. **Seller Dashboard** retrieves the new order and marks it as `packed`.
    4. **Delivery Dashboard** registers the order for shipping and delivery agents change status to `Delivered`.
    5. **Customer** checks their "Orders" history to see the package updated to `delivered`.

4. UAT (User Acceptance Testing)

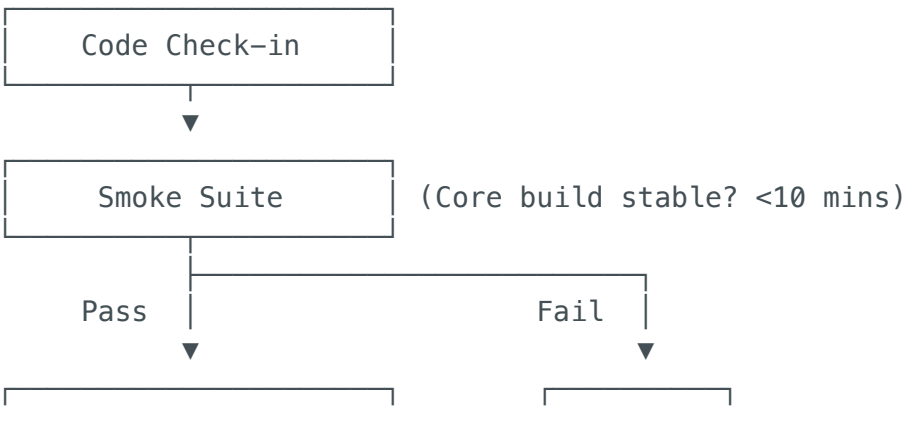
The final validation phase where product owners or target users run beta tests to confirm the product meets actual business requirements.

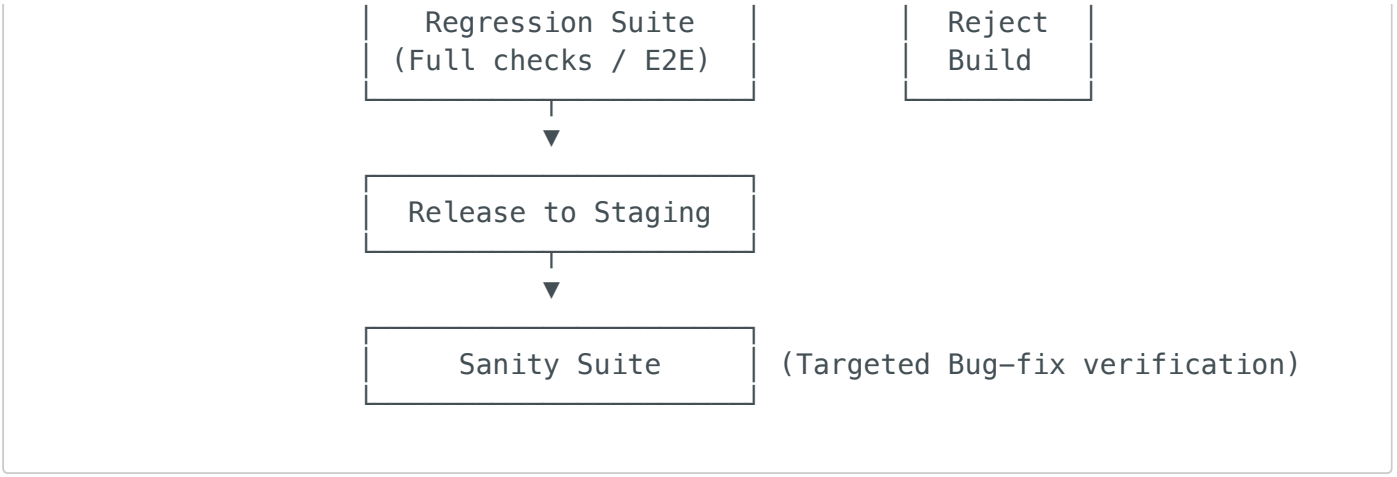
- 💡 **Sarathi Store Example:** Inviting mock shop owners to use the `SellerDashboard.tsx` UI to upload inventory items, checking if the navigation buttons and data charts match their day-to-day workflow.

---

10. Test Execution & Stability Suites

In continuous delivery, code changes are committed daily. Different suites are executed at different checkpoints:





1. Smoke Testing

A rapid suite of tests verifying that the primary, critical features of a new application build are functional and stable. If smoke tests fail, the build is instantly rejected.

- 💡 **Sarathi Store Smoke Test Cases:**
  1. Does the homepage load with a 200 HTTP code?
  2. Can users successfully log in with demo credentials?
  3. Does clicking "Add to Cart" successfully open the cart drawer?

2. Sanity Testing

A focused, quick test run verifying that a specific bug fix or minor patch works correctly without breaking immediately adjacent components.

- 💡 **Sarathi Store Example:**
  - **Bug:** Special characters like % or & inside the search bar cause the application to throw a 500 error page.
  - **Fix:** Special character encoding is added.
  - **Sanity Test:** QA runs 3-4 quick search inputs containing %, &, and ? to verify that the app no longer crashes.

3. Regression Testing

Re-running previous test suites (typically automated via Playwright) to confirm that new code additions or updates did not break existing features.

- 💡 **Sarathi Store Example:**

    - Developers add a **Product Review rating system** (`reviews.js`).
    - QA runs the **Cart & Checkout Regression suite** to confirm that introducing reviews did not break checkout logic or cart calculations.

4. Exploratory Testing

Unscripted, hands-on testing where the QA designs and executes test cases dynamically based on experience and system behavior. Excellent for finding edge-case defects that structured checklists miss.

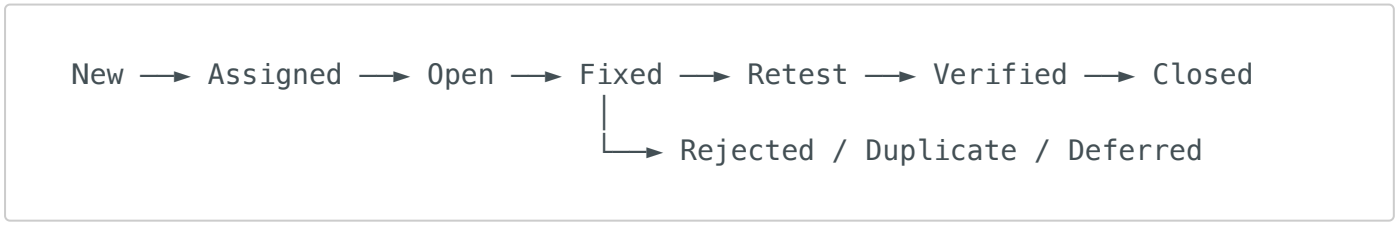
- 💡 **Sarathi Store Example:**
    - Testing checkout page validation and turning off local network connection exactly when clicking "Place Order" to check if the payment is processed twice or if the cart clears out incorrectly.

---

11. Defect Lifecycle & Management

Defect Lifecycle (Bug Lifecycle)

The sequence of states a software bug transitions through from its initial discovery by QA to its resolution and closure.



- **New:** QA finds a bug and logs it in Jira or GitHub Issues.
- **Assigned:** The team lead reviews and assigns the bug to a developer.
- **Open:** Developer is actively debugging and examining codebase files.
- **Fixed:** Developer fixes the code defect and pushes changes.
- **Retest:** QA pulls the updated build and re-executes the test case.
- **Verified:** QA confirms the bug is fixed.
- **Closed:** QA moves the ticket to closed state.
- **Rejected / Duplicate / Deferred:** Dismissed (not a bug), duplicate of another ticket, or postponed to a later release.

Severity vs. Priority

- **Severity:** The technical impact of the bug on system operations (defined by **QA**).
- **Priority:** The business urgency of fixing the bug (defined by **Product Owner**).

Severity / Priority	High Priority	Low Priority
High Severity	**Blocker / Core Path Failures:** Clicking "Place Order" results in a database crash and money is deducted without generating an order ID.	**Obscure Edge Case Crashes:** Generating a tax report CSV for a seller with exactly 0 sales in the year 2021 crashes the dashboard (critical bug, but rarely used).
Low Severity	**Branding / UX Flaws:** The store logo on the homepage is broken/missing, or the primary checkout button text says "Loginn" (low technical impact, high brand urgency).	**Minor Visual Misalignments:** The "Help" page footer copyright text is shifted by 3px on screen resolutions smaller than 360px.

Root Cause Analysis (RCA) - The 5 Whys

A problem-solving technique used to identify the core defect source to prevent it from happening again.

- 💡 **Scenario:** A customer was billed twice for a single order check-out.
  - *Why 1?* Two transaction rows were written to the Supabase database.
  - *Why 2?* The API received two POST requests to `/api/orders` within 200 milliseconds.
  - *Why 3?* The customer clicked the "Place Order" button twice rapidly.
  - *Why 4?* The "Place Order" button remained active after the first click.
  - *Why 5?* The frontend developer forgot to set the `submitting` state to `true` during order creation, which would disable the button. (**Root Cause**)

Session Summary & Interview Sheet

Key Takeaways

1. **Agile Framework:** Teams are formed into cross-functional pods where QA works alongside devs throughout the Sprint.
2. **STLC and SDLC:** SDLC focuses on building the code; STLC defines parallel validation quality checkpoints.
3. **EP and BVA:** Input testing must check partitions (valid vs. invalid) and boundary limits (Boundary, Boundary - 1, Boundary + 1) to optimize coverage.
4. **Smoke vs. Sanity vs. Regression:**
    - *Smoke:* Verifies build stability (broad and shallow).
    - *Sanity:* Verifies specific bug fixes (narrow and deep).
    - *Regression:* Verifies code updates didn't break old features (broad and deep).

Quick Interview Questions

- **Q: Difference between verification and validation?**
    - *A:* Verification checks if we built the product correctly (process-focused, QA). Validation checks if we built the right product (user-focused, QC/UAT).
- **Q: Can a bug be High Severity but Low Priority? Give an example.**
    - *A:* Yes. A page crash (High Severity) on an outdated seller terms and conditions page that is rarely accessed by users (Low Priority).
- **Q: When should automation script design start in a Sprint?**
    - *A:* During the Case Design phase, as soon as user stories and Acceptance Criteria (Given-When-Then) are defined. You do not wait for developer coding to finish before planning and draft-scripting Playwright locators.